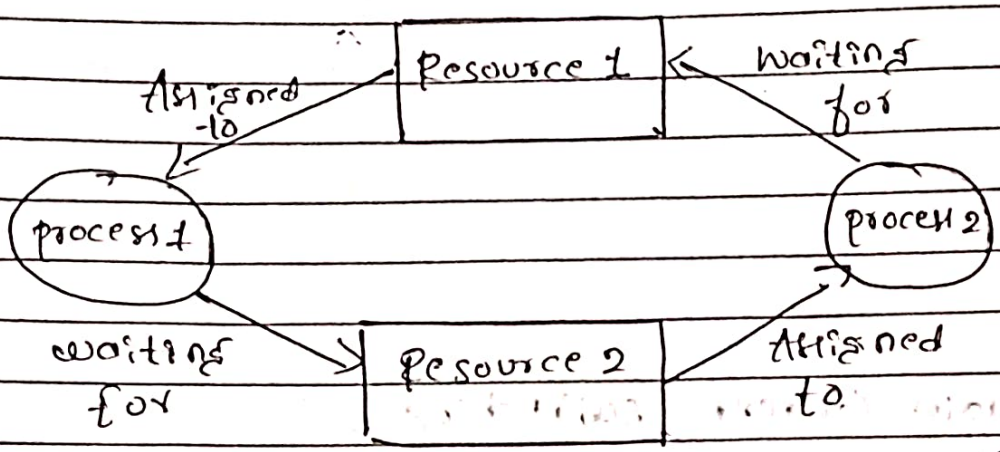
D D M M Y Y Y Y

Module - 3

5a) Define deadlock. what are the necessary conditions for deadlock to occur?

Deadlock is defined as a situation in which two or more competing actions are each waiting for the other to finish, preventing any action from taking place.



Necessary conditions for deadlock to occur:-
A deadlock situation can arise if the following four conditions hold simultaneously in a system:

1) Mutual Exclusion: This condition requires that at least one resource be non-sharable, meaning only one process can use it at a time; if another process requests the resource, it must wait until it's released.

2) Hold and wait: A process holding at least one resource waits to acquire additional resources held by other processes. this can create a situation where processes are indefinitely waiting for resources while holding onto others.

3) No preemption: Resources cannot be forcefully taken away from a process; they can only be released voluntarily by the process holding them, once its task is complete.

D D M M Y Y Y Y

5b) Illustrate peterson's solution for the critical section problem.

- peterson's solution:
- A classic software-based solution to the critical-section problem.
- Restricted to two processes, denoted as Po and P1, which alternate execution b/w their critical sections and remainder sections.

```
do {
        flag[i] = TRUE;
        turn=j;
        while (flag[j] && turn==j)
                ;//do nothing
        critical section
        flag[i] = FALSE;
                remainder section
} while (TRUE);
```

- Algorithm overview:

to enter the critical section, process Pi:

1. sets flag[i] to true.
2. sets turn to the value j, indicating that the other process [Pj] can enter its critical section.
3. If both process try to enter at the same time, turn will be set ~~on~~ to both i and j, but only one will last, ensuring mutual exclusion

D D M M Y Y Y Y

5c) Consider the following snapshot of the system:

	Allocation				MAX				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	8	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

Answer the following questions using Banker's algorithm:

a) What is the content of the need matrix?

b) Is the system in a safe state? If yes, mention the safe sequence.

If a request from process p1 arrives for (0,4,2,0) can the request be granted immediately?

Soln:-

Need = Max - Allocation

i) Need matrix

process	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	1	0	0	2
P3	0	0	2	0
P4	0	6	4	2

ii) To check whether the sim is in safety mode, we go with the safety algorithm.

=> P0 => 0000 ≤ 1520 [need ≤ available]

condition is satisfied,

1520 + 0012 = 1532 [available = available + allocation]

=> P1 => 0750 ≤ 1532

condition is not satisfied

D D M M Y Y Y Y

will break the deadlock cycle, but at great expense; the deadlocked processes may have computed for a long time, and the results of these partial computations must be discarded and probably will have to be recomputed later.

2) Abort one process at a time until the deadlock cycle is eliminated:
This method incurs considerable overhead, since after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any process is still deadlocked.

Factor influencing the selection of process for termination include:

1) Priority of the process.
2) Remaining computation time needed to complete its task.
3) Resources utilized by the process.
4) Additional resources required to finish
5) Number of process to be terminated

6b) What is a resource allocation graph? Consider an example to explain how it is every useful in describing a deadly embrace.

-> System resource-allocation graph consists of a set of vertices V and a set of edges E.
- V is partitioned into two types.
- P = {P1, P2 ..., Pn}
- R = {R1, R2 ... Rm},

D D M M Y Y Y Y

=> P2 => 1009 ≤ 1532
Condition is ~~not~~ satisfied
=> 1532 + 1334 = 2866
=> P3 => 0020 ≤ 2866
Condition is satisfied
2866 + 0632 = 2 14 11 8
=> P4 => 0642 ≤ 2 14 11 8
Condition is satisfied
2 14 11 8 + 0014 = 2 14 12 12
=> P1 => 0750 ≤ 2 14 12 12
Condition is satisfied
2 14 12 12 + 1000 = 3 14 12 12
Safe Sequence is:-
< P0, P2, P3, P4, P1 >

6) Explain different methods to recover from deadlocks.

RECOVERY FROM DEADLOCK

The system recovers from the deadlock automatically. There are two options for breaking a deadlock one is simply to abort one or more processes to break the circular wait. the other is to preempt ~~resi~~ some resources from one or more of the deadlock processes.

Process Termination:

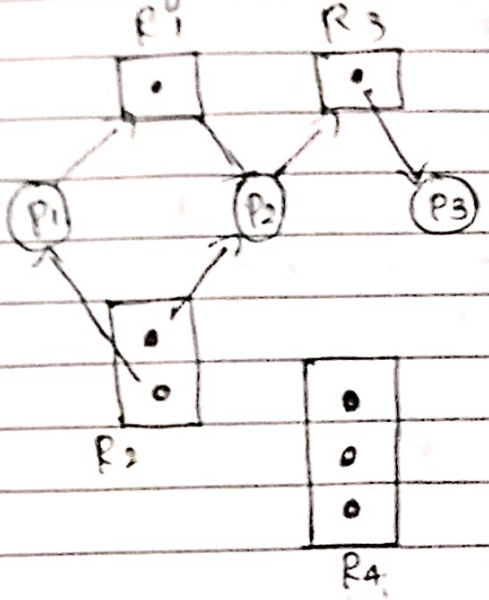
To eliminate deadlocks by aborting a process, we one of two methods In both methods, the system reclaims all resources allocated to the terminated processes.

1) Abort all deadlocked processes: This method clearly

A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$

A directed edge $P_i \rightarrow R_j$ is called a request edge, a directed edge $R_j \rightarrow P_i$ is called an assignment edge

The resource-allocation graph shown in below depicts the following situation



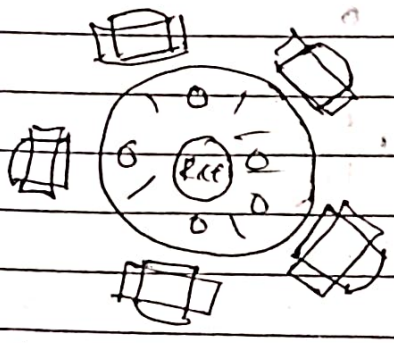
- Given the definition of a resource-allocation graph.
- If the graph does contain a cycle, then a deadlock may exist
- If each resource type has exactly one instance, then a cycle implies that a deadlock has occured.
- If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred.
- In this case, a cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.
- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occured.

D D M M Y Y Y Y

6c) What is semaphore? state a Dining philosophers problem gives a solution using semaphore

-> A semaphore is a synchronization tool is used solve various synchronization problem and can be implemented efficiently

The Dining philosophers problem is a classic synchronization problem where a set of philosophers sit around a table with each philosopher alternating b/w thinking and eating. the challenge arises from the need to prevent deadlock and starvation while allowing philosophers to access shared resources



1) Semaphore initialization: Initialize five semaphores, one for each chopstick. Initially, set the semaphore values to 1 to indicate that all chopsticks are available.

2) philosopher Actions: Each philosopher follows these steps:
- tries to pick up the two chopsticks closest to them, one at a time
- Eats if both chopsticks are acquired.

```
while (true)
{
    wait (chopstick[i]);
```

DD MM YYYY

```
wait (chopstick [(i+1)%5]);
  //eat
signal (chopstick[i]);
signal (chopstick [(i+1)%5]);
//thinks
}
```

3] preventing Deadlock:
Restrict the maximum number of philosophers allowed to be sitting simultaneously at the table to four. philosophers can pick up both chopsticks only if both are available.

Module-4

7a] what is TLB? Explain TLB in detail with a paging system with a neat diagram

TLB, or translation look-aside Buffer, is a hardware cache used to accelerate the translation of virtual addresses to physical addresses in a computer's memory management unit (MMU). Here's a detailed explanation of TLB with a paging system:

TLB overview:

- purpose: TLB serves as a fast lookup cache for translating virtual address to physical addresses, avoiding the slower access to the page table stored in memory.
- structure: It consists of key-value pairs, where the key represents the virtual address's page number, and the value contains the corresponding frame number in physical memory.

D D M M Y Y Y Y

TLB Working:

Address translation Process:

When the cpu generates a logical address, the TLB is presented with the page number portion of the address.

TLB HIT:

If the TLB contains an entry for the page number (TLB hit), the associated frame number is retrieved immediately, allowing direct access to the physical memory.

TLB MISS:

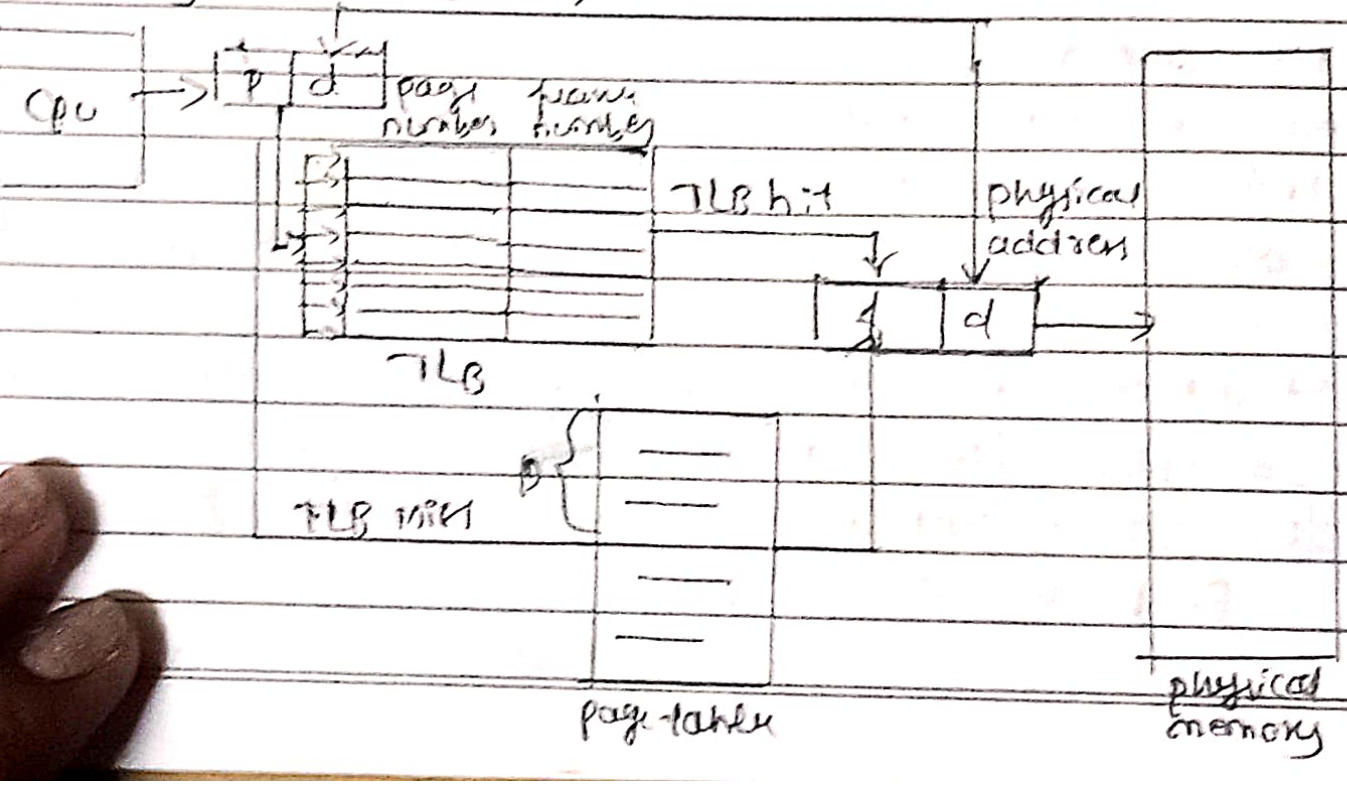
If the page number is not found in the TLB (TLB miss), a memory refrence to the page table in main memory is required to retrive the corresponding frame number.

TLB update:

Upon a TLB miss, the page number and its corresponding frame number are added to the TLB to expedite future translations.

Replacement policy:

If the TLB is full, the operating system selects an entry for replacement, often based on a least recently used (LRU)



D D M M Y Y Y Y

7b) With the help of a neat diagram, explain the various steps of address binding.

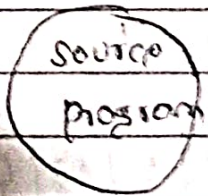
Address Binding :-

- User programs typically refer to memory addresses with symbolic names. Then symbolic names must be mapped or bound to physical memory addresses.

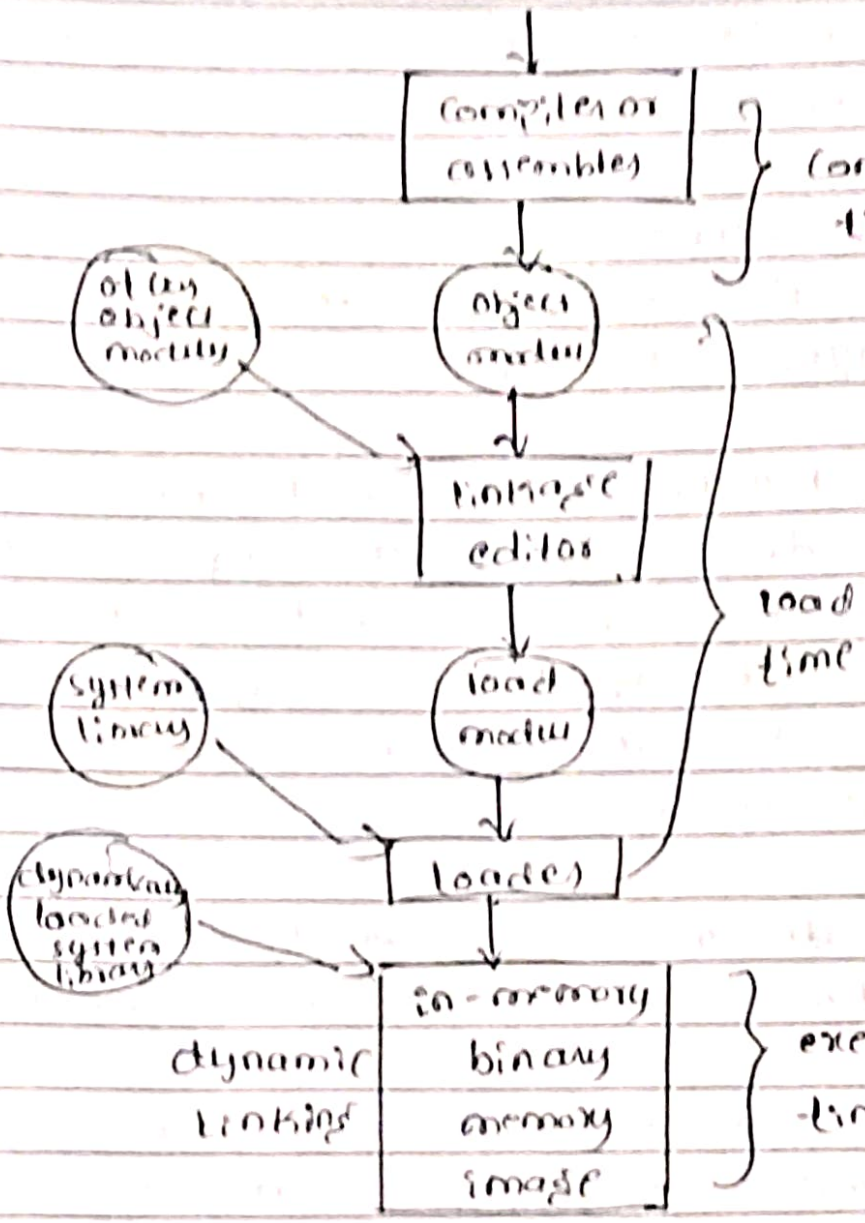
1) Compile time :- If it is known at compile time where a program will reside in physical memory, then absolute code can be generated by the compiler, containing actual physical addresses. However, if the load address changes at some later time, then the program will have to be recompiled.

2) Load time :- If the location at which a program will be loaded is not known at compile time, then the compiler must generate relocatable code, which references addresses relative to the start of the program. If the starting address changes, then the program must be reloaded but not recompiled.

3) Execution Time :- If a program can be moved around in memory during the course of its execution, then binding must be delayed until execution time.



D D M M Y Y Y Y



Consider the page reference string: 1,0,7,1,0,2,1,2,3,0,3, 2,4,0,3,6,2,1 for a memory with three frames. Determine the number of page faults using the FIFO, optimal, and LRU replacement algorithms. which algorithm is most efficient?

FIFO (replace the value used for long time)

1	0	7	1	0	2	1	2	3	0	3	2	4	0	3	6	2	1
1	1	1	1	(1)	2	2	2	(2)	0	0	0	0	0	3	3	3	1
	0	0	0	0	(0)	(1)	1	1	1	(1)	2	2	2	2	6	6	6
		7	7	7	7	7	(7)	3	3	3	(3)	4	4	4	4	2	2
*	*	*	hit	hit	*	*	hit	*	*	hit	*	*	hit	*	*	*	*



page (hit) : 5

page hit ratio = page hit / total no of reference string × 100

page fault (m) : 13

page fault ratio = page fault / total no of reference string × 100

LRU (Replace the value that is used less frequently before)

1	0	7	1	0	2	1	2	3	0	3	2	4	0	3	6	2	1
1	1	1	1	1	1	1	1	①	0	0	⑥	4	④	6	6	6	6
	0	0	0	0	0	0	⑤	3	3	3	3	③	0	⑥	2	2	2
		7	7	⑦	7	7	7	7	7	7	7	2	②	3	⑤	3	1
x	x	x	x	hit	x	hit	hit	x	x	hit	hit	x	x	hit	hit	hit	x

page hit (hit) : 9

page hit ratio = page hit / total no of reference string × 100

page fault (x) : 9

page fault ratio = page fault / total no of reference string × 100

page hit ratio : (9/18) × 100 = 50%

page fault ratio = (9/18) × 100 = 50%

Optimal (replace value that is not used for longest period in future)

1	0	7	1	0	2	1	2	3	0	3	2	4	0	3	6	2	1
1	1	1	1	1	1	1	①	⑧	3	3	3	3	3	3	6	6	6
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	2	2
		7	7	⑦	7	7	7	7	7	7	②	4	4	4	4	4	1
x	x	x	hit	hit	x	hit	hit	x	hit	hit	x	x	hit	hit	x	x	x

page hit (hit) : 9

page hit ratio = page hit / total no of reference string × 100

page fault (x) = 9

page fault ratio = page fault / total no. of reference string × 100

page hit ratio : (9/18) × 100 = 50% page fault ratio = (9/18) × 100 = 50%

D D M M Y Y Y Y

what is demand paging? Explain the steps in handling page faults using the appropriate diagram

Demand paging:-

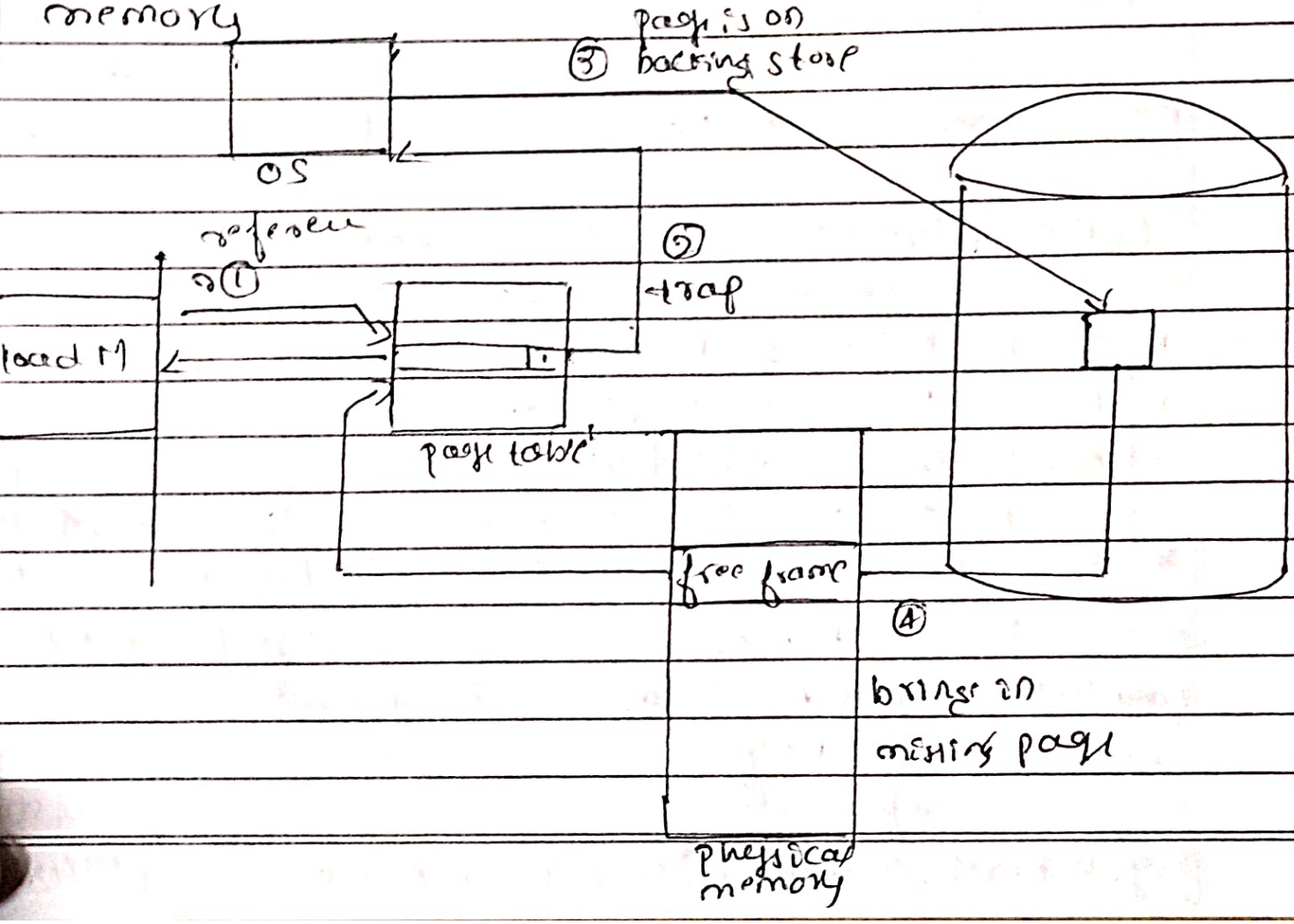
A demand paging is similar to paging system with swapping when we want to execute a process we swap the process in to memory otherwise it will not be loaded in to memory. page fault

If a page is needed that was not originally loaded up, then a page fault trap is generated

steps in handling page fault:-

The memory address requested is first checked, to make sure it was a valid memory request

If the reference is to an invalid page, the process is terminated. otherwise, if the page is not present in memory



D D M M Y Y Y Y

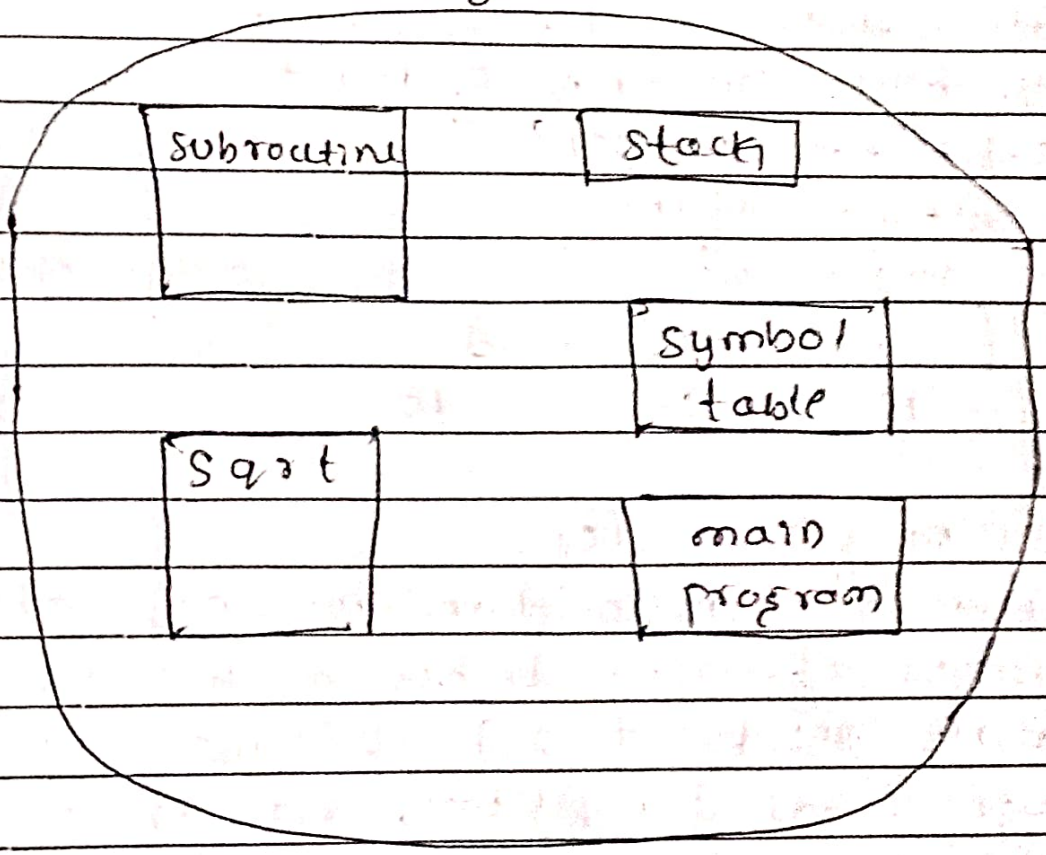
8b) what is segmentation? Explain the basic method of segmentation with an example

Segmentation:

- segmentation organizes memory into segments, each with a unique name and length, providing a more flexible memory-management scheme.

Basic methods of segmentation:

1) memory is divided into logical address spaces, comprising segments identified by unique names
2) each segment represents a distinct part of the program,
3) such as code, global variables, heap, stack, etc
4) During compilation, the compiler automatically constructs segments based on the program's structure.



Logical address

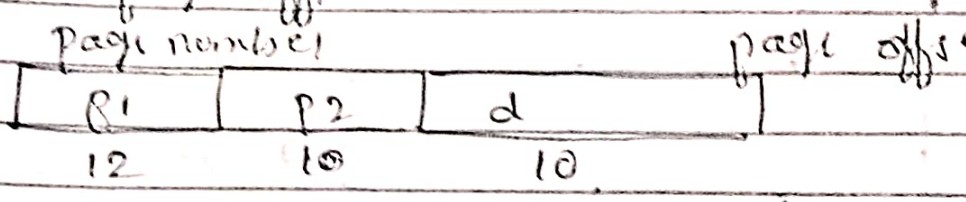
D D M M Y Y Y Y

8c) Discuss the structure of the page table with a suitable diagram

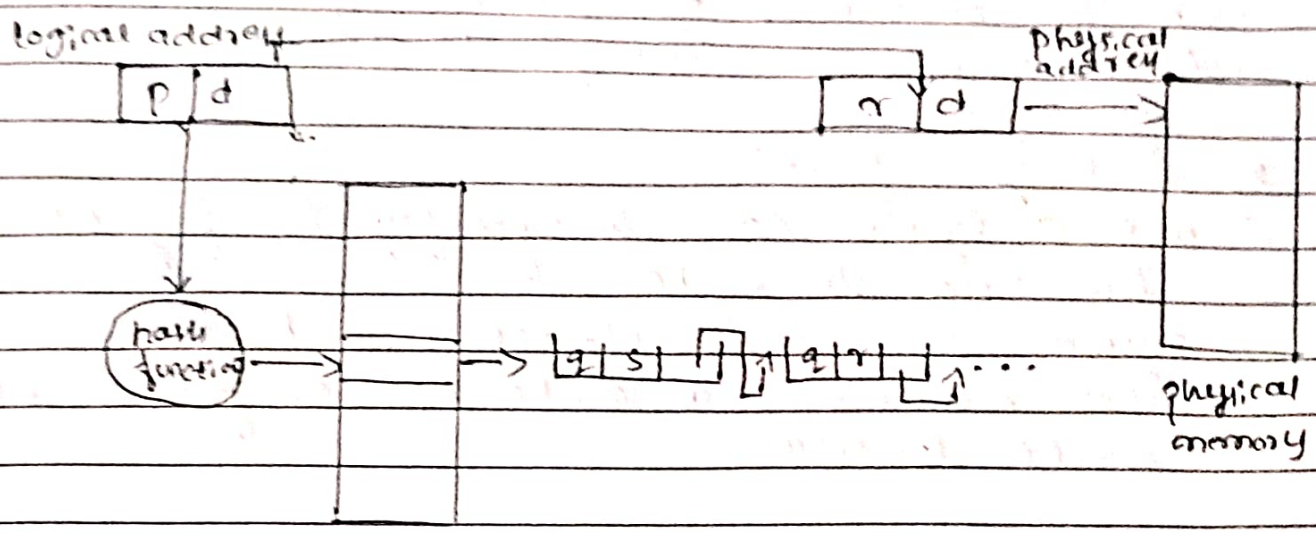
- hierarchical paging:
- problem & solution: large logical address space necessitates smaller page tables. Hierarchical paging divides the page table into smaller sections to address this issue
- usage: common in systems with extensive logical address spaces.
- mechanism: page table is structured hierarchically, reducing individual page table sizes.

A logical address (on a 32-bit machine with a 1K page size) is divided into:

- page number consisting of 22 bits
- page offset consisting of 10 bit
- 12-bit page number
- 10-bit page offset



- hashed page tables:
- problem & solution: handling large address spaces requires efficient lookup mechanism. Hashed page tables resolve this by hashing virtual page numbers.
- usage: Ideal for systems with extensive address spaces requiring efficient virtual-to-physical mapping
- mechanism: Hashed value represent virtual page numbers with collision handling using linked lists
- description: Hashing virtual page numbers enables



Module - 5

9a) What is file? What are its attributes? Explain file operations.

A file is a named collection of related information recorded on secondary storage, defined by the creator which can encompass various types

File Attributes are :-

1. Name
2. Identifier
3. Type
4. Location
5. Size
6. Protection
7. Time, date and user identification

File operations are

1) creating a file :-

- Allocate space in the file system and create an entry

D D M M Y Y Y Y

in the directory for the new file.
- Assign a unique identifier to the file within the file system

2) Writing a file:-
- use a system call to specify the file name and the data to write
- System searches the directory to locate the file and maintains a write pointer for the next write location

3) Reading file:
use a system call to specify the file name and where the read data.
- System searches the directory for the file and maintains a read pointer

4) Deleting a file:-
- search the directory for the file and erase its entry.
- Release file space for reuse by other files.

5) Error handling:
- System calls return error codes to indicate file operation success or failure
- Error may occur due to insufficient permissions, disk full, or other issues.

6) File system consistency:
- Ensure that file system data structures remain intact and coherent.

Explain contiguous and linked disk space allocation methods

Contiguous Allocation:

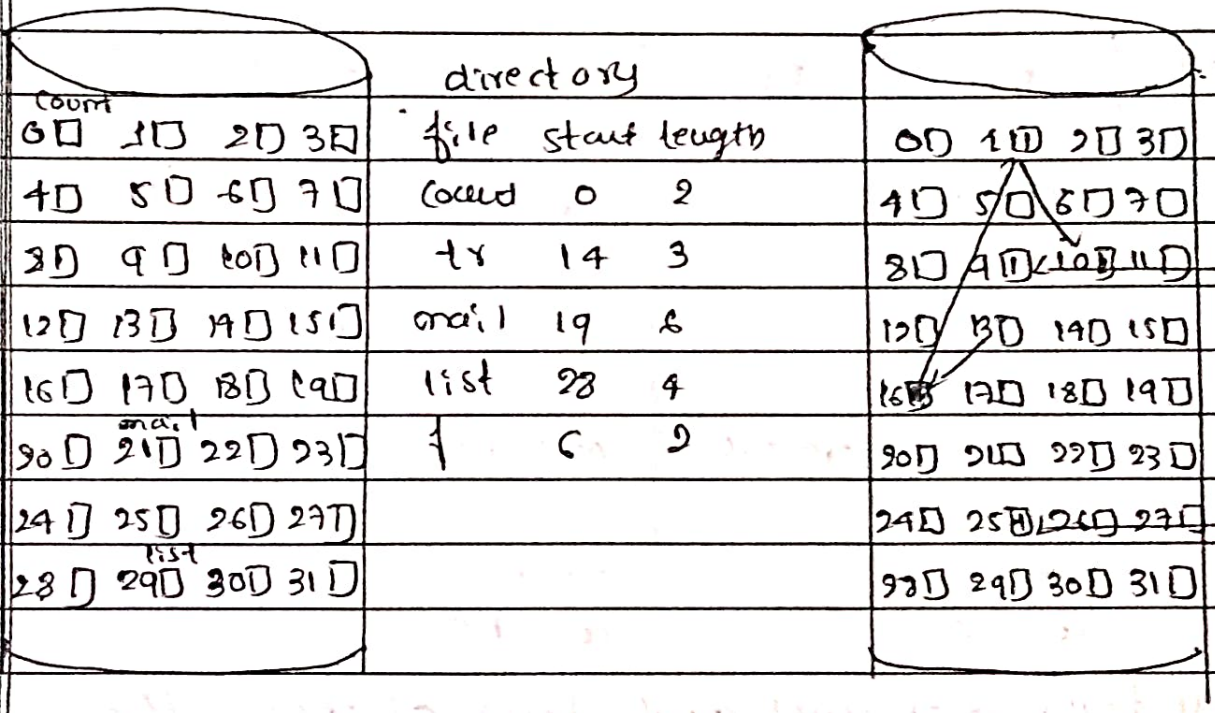
Definition: Files are stored in contiguous blocks on the disk, occupying sequential disk space.

Accessing files: simple access with starting block number and length required

Access Efficiency: supports efficient sequential and direct access operations.

Challenges:

Finding space: Difficulty to find contiguous space for new files, depending on the free space management system.



Linked Allocation:-

Definition: files are stored as linked lists of disk blocks, with blocks scattered across the disk.

File Structure: Each file represented by a linked list of blocks, with directory entries containing pointers to

3) File creation - creating a new file involves adding an entry in the directory with pointers initialized to nil.

10) Explain ~~configuration~~ the access matrix method of system protection with the domain as objects and its implementation.

Access matrix

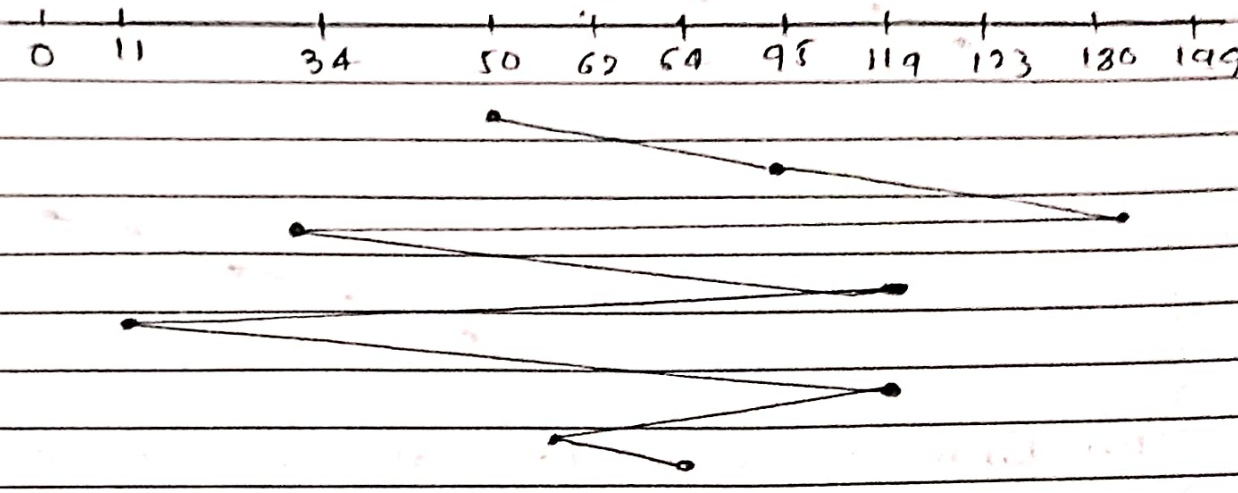
- The model of protection can be viewed abstractly as a matrix, called access matrix
- The entry access(i,j) defines the set of operation that a process executing in domain Di can invoke on object Oj.

Object domain	F1	F2	F3	printer
D1	read		read	
D2				print
D3		read	execute	
D4	read write		read write	

- There are four domains and four objects - three files (F1, F2, F3) and one laser printer. A process executing in domain D1 can read files F1 and F3.
- A process executing in domain D4 has the same privileges as one executing in domain D1; but in addition, it can also write onto files F1 and F3.

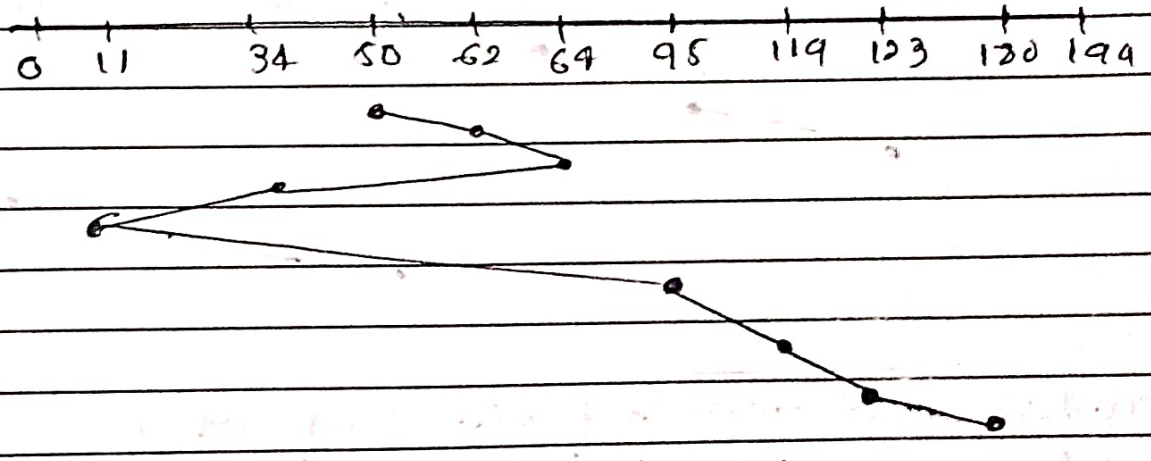
10c) Given the following sequences 95, 180, 34, 119, 11, 123, 62, 64 with the track 50 and ending track, lay extent is the total disk traversed by the disk arm using FCFS, SSTF, look algorithm

(FCFS)



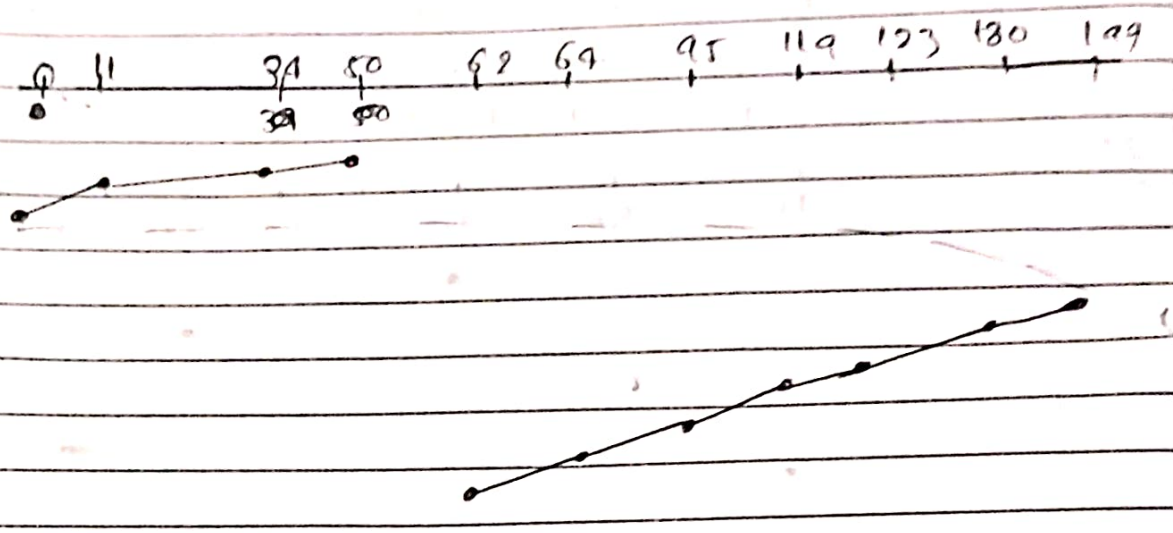
total head movement: 640 tracks

(SSTF)



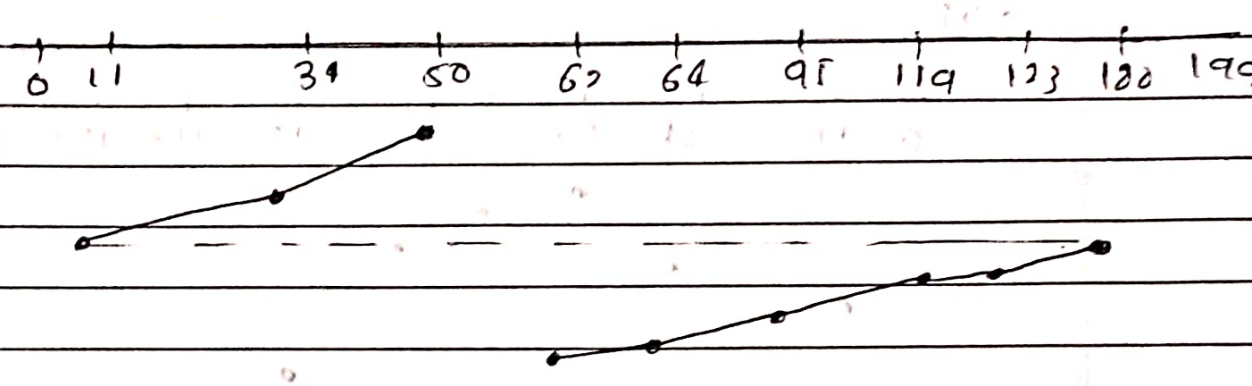
- total head movement: 236 tracks.

(C- Scan)



- The total head movement is relatively low, only 187 tracks.

(C- LOOK)



- Total head movement further compared to C-SCAN, down to 157 tracks.